

JCMOVES Compliant Rewards System

Replit-Ready Folder Structure

Printable implementation blueprint with exact file paths, imports, and startup steps

Build Goal

Service marketplace + off-chain rewards ledger + optional self-custody wallet connection + sweepstakes-style engagement

1. Stack Assumptions

- Frontend: React / Vite in client/
- Backend: Express / TypeScript in server/
- Database: Neon Postgres + Drizzle in db/
- Shared configs and types in shared/
- Rewards are off-chain by default; no custody, no internal trading

2. Folder Structure

```
root/
├── client/
│   ├── src/
│   │   ├── components/
│   │   │   └── rewards/
│   │   │       ├── RewardsBalanceCard.tsx
│   │   │       ├── RewardsRedeemGrid.tsx
│   │   │       ├── RewardsLedgerList.tsx
│   │   │       └── WalletNotice.tsx
│   │   ├── lib/
│   │   │   └── api.ts
│   │   ├── pages/
│   │   │   └── Rewards.tsx
│   │   ├── App.tsx
│   │   ├── main.tsx
│   │   └── index.css
│   └── index.html
├── server/
│   ├── config/
│   │   └── rewards.ts
│   ├── lib/
│   │   ├── idempotency.ts
│   │   └── errors.ts
│   ├── routes/
│   │   ├── jobs.ts
│   │   ├── rewards.ts
│   │   └── sweepstakes.ts
│   ├── services/
│   │   ├── rewardsService.ts
│   │   └── sweepstakesService.ts
│   ├── db.ts
│   └── index.ts
├── db/
│   ├── index.ts
│   ├── schema.ts
│   └── migrate.ts
├── shared/
│   ├── rewards.ts
│   └── types.ts
└── .env
```

```
├─ drizzle.config.ts
├─ package.json
├─ tsconfig.json
└─ vite.config.ts
```

3. Key Files and Purpose

File	Purpose
shared/rewards.ts	Central rewards catalog and point rules shared by frontend and backend.
db/schema.ts	Drizzle tables for users, jobs, reward ledger, redemptions, and sweepstakes entries.
server/services/rewardsService.ts	Awards rewards on job completion and handles redemptions.
server/routes/jobs.ts	Creates jobs and marks them completed.
server/routes/rewards.ts	Returns balance, ledger history, and redemption endpoints.
server/routes/sweepstakes.ts	Handles active round lookup and no-purchase free entry.
client/src/pages/Rewards.tsx	User-facing rewards dashboard.

4. Exact Import Patterns

Server files

```
import { db } from "../../db";
import { jobs, users } from "../../db/schema";
import { REWARDS } from "../../shared/rewards";
```

Client files

```
import type { RewardBalanceResponse } from "../../shared/types";
import { REWARDS } from "../../shared/rewards";
```

5. Environment Variables

```
DATABASE_URL=your_neon_database_url_here
PORT=5000
VITE_API_BASE=http://localhost:5000
```

6. Replit Startup Steps

1. npm install
2. npm run db:push
3. npm run dev
4. Replace the placeholder user ID in client/src/pages/Rewards.tsx with your real auth user ID or a seeded test user.

7. Compliance Guardrails

- Use the system as a service rewards platform, not a trading platform.
- Do not custody user crypto, private keys, or seed phrases.
- Do not market JCMOVES as an investment or promise returns.
- Keep any wallet integration optional and self-custodial.
- Use sweepstakes-style entries with a no-purchase free entry path.

8. Next Integration Target

Wire the rewards service into your real jobs pipeline so rewards are issued automatically when an admin marks a job completed. That is the highest-value next patch after this printable blueprint.